

진짜 서비스 만들기 — 심리테스트 공유 서비스

GitHub·Vercel·Supabase 3종 세트 · 강사 진행 대본 (정금구)

📖 사용법

한 페이지 = 한 슬라이드. ★ KEY POINT(보라) = 반드시 전달할 한 문장 · 🔍 화면(파랑) = 이 슬라이드에 띄울 것 · 💬 이렇게 말하세요(검정) = 실제 멘트 · 💡 강사 팁(노랑) · 📌 참고(회색) · → 넘어가며.
구성: 이론부(슬라이드 1~23) + 실습부(24~29) + 마무리(30~31). ✂️ 2회 분할 시 23 끝에서 끊는다.

📚 용어 한 줄 (강사용)

5요소 = 몸(앱 코드) · 기억(GitHub) · 무대(Vercel) · 창고(Supabase) · 맥박(push → 자동배포)
클라이언트/서버 손님/주방 · 클라우드 빌려 쓰는 남의 컴퓨터 · 커밋/push 세이브/올리기 · 배포 무대에 올리기 · CI/CD
push하면 알아서 재배포되는 컨베이어 벨트 · 테이블 규칙 있는 엑셀 시트 · SQL 창고에 말 거는 언어 · anon 키 손님용 방
문증(공개 OK) · RLS 문지기 규칙표

구분	시간	슬라이드
Part 0 — 오프닝 + 라이브 시연	8분	1~4
이론 ① — 앱과 서비스 (손님·주방·클라우드)	10분	5~8
이론 ② — GitHub 기억 (버전 관리)	12분	9~12
이론 ③ — Vercel 무대 (배포·자동배포)	12분	13~16
이론 ④ — Supabase 창고 (DB·SQL·보안)	15분	17~21
이론 ⑤ — 총정리 ✂️ 분할점	8분	22~23
실습 P6 — 내 테스트 기획 (Claude 대화)	15분	24
실습 P7 — 백엔드 키트 위에 바이브 코딩 ★	30분	25
실습 P8 — GitHub 입주	12분	26
실습 P9 — Vercel 무대	10분	27
실습 P10 — Supabase 창고 ★	25분	28
실습 P11 — 맥박 + 공유 이벤트 ★	13분	29
마무리	10분	30~31

표지 — "내 컴퓨터를 꺼도 서비스는 살아있다"

01분

★ KEY POINT

오늘은 앱이 아니라 '서비스'를 만든다. 코드·실행·데이터에게 각자 집을 주는 날.

■ 화면

표지 — 강의명 + 우측 5요소(몸·기억·무대·창고·맥박).

💬 이렇게 말하세요

안녕하세요. 오늘은 시리즈에서 제일 큰 걸음을 내딛는 날입니다. 지금까지 우리가 만든 건 전부 '앱'이었어요. 오늘 만드는 건 '서비스'입니다. 뭐가 다르냐 — 제 컴퓨터를 꺼도 살아있고, 여러분 친구 폰에서도 돌아간다는 겁니다. 오른쪽 다섯 단어, 오늘 끝나면 전부 여러분 것이 됩니다.

→ 넘어가며: "먼저 — 7차에 만든 그 앱, 지금도 살아있나요?"

그 앱, 지금도 살아있나요?

03분

★ KEY POINT

데이터가 '내 기기 안'에 사는 한, 그건 서비스가 아니라 앱이다.

■ 화면

살아있는 것 ✓ vs 갇혀 있는 것 ✕ 2단 비교.

💬 이렇게 말하세요

7차에 만든 할 일 앱, URL은 지금도 살아있죠. 근데 질문 하나 — 여러분이 거기 적은 할 일은 지금 어디 있죠? 그 폰, 그 브라우저 안에만 있습니다. localStorage — '저장'의 '저'였죠. 폰을 바꾸면? 사라집니다. 친구한테 보여주려면? 방법이 없습니다. "우리 앱 사용자들이 뭘 제일 많이 적을까?" 같은 통계는? 원리적으로 불가능합니다. 데이터가 각자 기기에 갇혀 있으니까요.

💡 강사 팁

실제로 7차 앱을 띄워 "제 폰엔 이게 있는데 여러분 폰엔 뭐가 있죠?"라고 물으면 한 번에 체감된다.

→ 넘어가며: "그럼 오늘 뭘 만드는지 — 완성본부터 보여드릴게요."

라이브 시연 — 심리테스트 공유 서비스

03분

★ KEY POINT

친구 폰에서 내 결과가 열린다 — 이게 되려면 뭐가 필요한가? 그게 오늘의 이론.

■ 화면

시연 타임라인 4단계. 실제 시연은 강사 폰 + 완성본(참고샘플 배포본)으로.

💬 이렇게 말하세요

(폰으로 완성본 열며) 오늘 여러분이 만들 서비스입니다. 질문 다섯 개 풀면 — 저는 '충직한 강아지'가 나왔네요. 그리고 여기, 전체 참여자 중 저 같은 유형이 몇 퍼센트인지도 나옵니다. 자 이제 핵심 — 이 '공유 링크 복사' 버튼을 누르고, ○○님께 카톡으로 보낼게요. ○○님, 열어보세요. 뭐가 보이죠? 네, 제 결과가 그대로 보입니다. ○○님은 테스트를 푼 적도 없는데요. 오늘 수업 전체가 이 한 장면을 설명하고, 여러분 손으로 재현하는 겁니다.

💡 강사 팁

시연용 완성본은 수업 전에 미리 배포해두고 링크를 준비. 이 "카톡 한 방" 장면이 오늘 최고의 후킹 — 서두르지 말 것.

→ 넘어가며: "오늘 일정, 이렇게 갑니다."

오늘의 약속 — 이론 → 실습 → 결과물

01분

★ KEY POINT

전반전은 원리(세 집의 이론), 후반전은 입주(실습). 결과물은 내 서비스 URL.

■ 화면

이론·실습·결과물 3카드 + 5요소 콜아웃.

💬 이렇게 말하세요

오늘은 이론을 제대로 합니다. 도구 셋을 '따라 하기'로만 배우면 다음에 혼자 못 하거든요. 전반전에 세 집의 원리 — GitHub, Vercel, Supabase가 왜 필요하고 뭘 하는 물건인지 — 를 다지고, 후반전에 여러분이 직접 기획한 테스트를 그 세 집에 입주시킵니다.

→ 넘어가며: "이론 첫 번째 — 인터넷은 손님과 주방입니다."

인터넷은 손님과 주방이다 (클라이언트·서버)

03분

★ KEY POINT

인터넷의 모든 일 = 요청하는 손님(클라이언트) + 응답하는 주방(서버). 지금까지 우리는 '내가 손님이자 주방'이었다.

■ 화면

손님(📱) ↔ 주방(👨‍🍳) 그림.

💬 이렇게 말하세요

인터넷이 돌아가는 원리, 사실 식당이랑 똑같습니다. 여러분이 폰으로 유튜브를 열면 — 폰이 손님이에요. "이 영상 주세요"라고 주문(요청)하죠. 그러면 어딘가에 있는 24시간 켜진 컴퓨터, 즉 주방(서버)이 영상을 내웁니다(응답). 이 두 역할이 전부예요. 그런데 지금까지 우리가 만든 앱들은 — 4차 Artifacts도, 7차 localhost도 — 내 기기 안에서 내가 손님이자 주방이었습니다. 자급자족이요. 그래서 남이 못 왔던 겁니다.

→ 넘어가며: "그 'localhost'라는 주소, 정체는 뭡가요?"

localhost의 정체 — "내 컴퓨터 안"

⌚ 2분

★ KEY POINT

남이 오려면 ① 24시간 컴퓨터 ② 전 세계가 찾는 주소 ③ HTTPS — 셋 다 내 노트북엔 없다. 그래서 빌린다.

■ 화면

필요 3가지 카드 + "그래서 — 빌린다" 큰 글씨.

💬 이렇게 말하세요

localhost는 '내 컴퓨터 안'이라는 뜻의 예약된 주소입니다. 옆 사람 폰에서 그 주소를 치면 '옆 사람의 컴퓨터 안'을 찾아요. 그러니 안 열리는 게 당연하죠. 남들이 오게 하려면 세 가지가 필요합니다. 24시간 안 꺼지는 컴퓨터, 전 세계 어디서든 찾아올 수 있는 주소, 그리고 안전한 연결 — 7차에서 "설치는 HTTPS에서만 된다"고 했던 그 자물쇠요. 셋 다 내 노트북엔 없습니다. 그래서 — 빌립니다.

→ 넘어가며: "빌려주는 회사들이 있거든요. 그게 클라우드입니다."

클라우드 = 남의 컴퓨터 빌려 쓰기 (3회사 표)

⌚ 3분

★ KEY POINT

셋 다 클라우드 회사 — 단, 빌려주는 게 다르다. 기억(GitHub)·무대(Vercel)·창고(Supabase).

■ 화면

3회사 비교 표 + "셋 다 무료, 셋 다 GitHub 계정으로 로그인".

💬 이렇게 말하세요

'클라우드'라는 말, 뉴스에서 많이 들으셨죠. 별거 아닙니다 — 데이터센터에 컴퓨터를 수만 대 켜놓고 빌려주는 사업이에요. 오늘 쓰는 세 회사도 전부 클라우드 회사입니다. 다만 빌려주는 물건이 달라요. GitHub는 코드 보관용 컴퓨터 — 기억. Vercel은 앱 실행용 컴퓨터에 주소와 자물쇠까지 — 무대. Supabase는 데이터 보관용 컴퓨터에 관리 화면까지 — 창고. 그리고 재밌는 게, 셋 다 무료로 시작하고 셋 다 GitHub 계정 하나로 로그인됩니다. GitHub가 개발 세계의 신분증이란 말이 그래서 나옵니다.

📌 참고

숙제로 계정 3개 가입을 미리 시켰다는 전제. 안 한 수강생은 쉬는 시간에 지원.

→ 넘어가며: "근데 왜 하나로 안 하고 셋으로 나눌까요?"

왜 셋으로 나누나 — 한 집 = 한 책임

⌚ 2분

★ KEY POINT

성격이 다르면 집도 다르다. 나눠 놓으면 갈아끼울 수 있다 — 8차 L(나눈다)의 서비스판.

■ 화면

3카드(한 집=한 책임 · 갈아끼우기 · 채점기도 같은 구조).

💬 이렇게 말하세요

코드는 '역사'가 중요하고, 실행은 '안 꺼짐'이 중요하고, 데이터는 '안 잃어버림'이 중요합니다. 성격이 다르니 집도 다른 거예요. 8차 들으신 분들, 익숙하죠? Layer — 나눈다. 나눠 놓으면 나중에 무대를 옮겨도 기억과 참고는 그대로입니다. 그리고 이거, 장난감 구조가 아니에요. 여러분이 쓰는 이 채점기 리더보드도 코드는 GitHub, 무대는 Railway, 참고는 Supabase — 오늘 배우는 것과 똑같이 삽니다.

→ 넘어가며: "자, 첫 번째 집부터 — 기억, GitHub."

'최종_진짜_최종.js'의 지옥

⌚ 3분

★ KEY POINT

GitHub는 백업집 + 역사책 + 접선 장소. 특히 '접선 장소'가 없으면 자동배포 자체가 성립 안 한다.

■ 화면

없으면 ✕ vs 있으면 ✓ 2단.

💬 이렇게 말하세요

코드가 내 폴더에만 있으면 무슨 일이 생기냐. 첫째, 노트북이 고장 나면 서비스가 통째로 증발합니다. 둘째, '어제까진 됐는데' 상태로 못 돌아갑니다 — 6차 때 정리했던 '회의록 진짜최종.docx' 지옥이 코드에서 재현되는 거죠. 셋째, 이게 오늘의 핵심인데 — Vercel이 내 코드를 가져가려면 '공식 주소'가 있어야 해요. GitHub가 없으면 자동배포라는 개념 자체가 성립을 안 합니다. GitHub는 백업집이자 역사책이면서, 다른 서비스들이 내 코드를 데려가는 접선 장소입니다.

→ 넘어가며: "이 '역사를 남기는 기술'이 버전 관리 — 게임 세이브랑 똑같습니다."

버전 관리 = 게임 세이브

⌚ 4분

★ KEY POINT

Git=게임기(세이브 프로그램) · GitHub=클라우드 세이브 · 커밋=세이브 1회 · push=올리기.

■ 화면

Git/GitHub/커밋/push/브랜치 정의 리스트.

💬 이렇게 말하세요

게임 하면서 세이브 안 하는 분 없죠. 버전 관리가 정확히 그겁니다. Git이라는 프로그램이 내 컴퓨터에서 도는 세이브 기능이에요 — 게임기라고 생각하세요. '커밋'이 세이브 한 번입니다. 지금 상태의 스냅샷에 "뭘 바꿨는지" 메모를 붙여 저장하는 거예요. 이 메모가 중요합니다 — 미래의 내가 읽을 쪽지거든요. 그리고 'push'가 그 세이브들을 GitHub, 즉 클라우드 세이브 서버에 올리는 겁니다. push해야 비로소 내 컴퓨터 밖에 존재하는 거예요. 브랜치라는 것도 있는데 — 세이브를 복제해서 딴 길로 가보는 기능 — 오늘은 본선 하나만 씁니다.

💡 강사 팁

"커밋 = 세이브, push = 클라우드 업로드" 이 두 등식만 각인되면 이 파트는 성공.

→ 넘어가며: "**GitHub에 뭐가 있는지 지도를 한번 펼쳐볼게요.**"

GitHub 기능 지도

0 2분

★ KEY POINT

오늘 쓰는 건 repo·커밋·push·히스토리. public repo는 그대로 포트폴리오가 된다.

■ 화면

기능 표 6줄.

💬 이렇게 말하세요

repo는 프로젝트 하나가 사는 방입니다. 만들면 github.com/내아이디/프로젝트명 — 주소가 생겨요. 히스토리에선 모든 커밋이 시간순으로 보이고, 커밋 하나를 클릭하면 지운 줄은 빨갭게, 넣은 줄은 초록으로 보여줍니다. README는 4차에서 배운 '사람한테 한 장' — repo의 문패고요. 그리고 public으로 두면 누구나 볼 수 있는데, 이게 그대로 포트폴리오입니다. 요즘 개발자 이력서엔 GitHub 주소가 들어가요. Issues니 PR이니 하는 건 회사에서 협업할 때 쓰는 결재 라인 — 오늘은 안 씁니다.

→ 넘어가며: "그래서 오늘 우리가 쓸 동작은 — 딱 3개."

오늘 쓰는 동작은 딱 3개

03분

★ KEY POINT

명령어 암기 X — Claude Code에게 말로 시킨다. 로그인 / 새 저장소로 올리기 / 커밋하고 push.

■ 화면

3동작 스텝.

💬 이렇게 말하세요

git 명령어를 외울 필요가 없습니다. 우리는 바이브 코딩 시리즈답게 Claude Code에게 말로 시킵니다. 처음 한번 "GitHub에 로그인해줘" — 브라우저 인증만 직접 하시면 되고요. 그다음 "이 폴더를 새 저장소로 올려줘" — 이것도 한 번이면 끝. 그 뒤로는 뭔가 고칠 때마다 "커밋하고 push해줘" 이 한 마디의 무한 반복입니다. 대신 뒤에서 무슨 일이 일어났는지는 알아야죠 — 그래서 매번 웹에서 커밋 목록과 빨강/초록 비교 화면을 확인할 겁니다.

→ 넘어가며: "기억 완성. 이제 두 번째 집 — 무대."

7차 배포는 택배, 오늘은 컨베이어 벨트

⌚ 3분

★ KEY POINT

Vercel = 24시간 극장. 7차 CLI는 택배 한 번 — 오늘은 GitHub와 이어 컨베이어 벨트를 간다.

■ 화면

7차 CLI ✕ vs GitHub 연동 ✓ 2단.

💬 이렇게 말하세요

7차에서 배포 해봤죠 — `vercel --prod`. 그건 완성본을 택배로 한 번 부치는 방식이었어요. 고칠 때마다 다시 부쳐야 했죠. 사람이 손으로 하는 배포는 잊어버리고, 늦고, 실수합니다. 오늘은 방식을 바꿉니다. 무대(Vercel)를 기억(GitHub)에 한 번 연결해두면 — push할 때마다 무대가 알아서 새 버전을 가져가 올립니다. 컨베이어 벨트예요. 현업에서 CI/CD라고 부르는 표준을 우리가 그대로 까는 겁니다.

→ 넘어가며: "배포하면 자동으로 따라오는 선물이 세 개 있습니다."

배포의 3종 자동 선물 — 주소·자물쇠·간단함

03분

★ KEY POINT

배포하면 전 세계 유일 주소 + HTTPS 자물쇠가 자동. 우리 앱은 정적(도시락)이라 설정도 없다.

■ 화면

3선물 카드.

💬 이렇게 말하세요

배포하면 우선 내앱.vercel.app — 전 세계에서 유일한 주소가 생깁니다. 그리고 주소창의 자물쇠, HTTPS가 자동으로 채워져요. 7차에서 "설치랑 오프라인은 HTTPS에서만 된다"고 했던 그 자물쇠입니다. 마지막으로 — 우리 앱은 '정적 사이트'예요. 주방에서 조리가 필요 없는 도시락. 파일을 그대로 내주면 되는 앱이라 배포 설정이랄 게 없습니다. 조리가 필요한 앱은 설정이 더 붙는데, 그건 나중 이야기.

📌 참고

환경변수 질문이 나오면: "비밀 값을 코드 밖에 두는 금고 — 오늘은 필요 없는 이유를 참고 열쇠 이론에서 설명합니다."

→ 넘어가며: "그 컨베이어 벨트가 실제로 어떻게 도는지 볼까요?"

자동배포의 흐름, 그리고 롤백

03분

★ KEY POINT

push → Building → Ready. 망가지면 클릭 한 번 롤백 — "무서워서 못 고친다"가 사라진다.

■ 화면

타임라인 4단계.

💬 이렇게 말하세요

흐름은 이렇습니다. 내가 하는 건 push 하나. 그러면 Vercel이 감지하고 Building — 새 버전을 준비합니다. 몇십 초 뒤 Ready — 같은 주소에 새 버전이 전 세계 공개. 그리고 배포도 커밋처럼 역사가 남아요. 배포 목록의 각 줄에 커밋 메시지가 그대로 붙습니다. 새 버전이 망가졌다? 클릭 한 번이면 직전 배포로 돌아갑니다 — 롤백. 커밋으로 되돌리고 롤백으로 되돌리고 — 이중 안전장치가 있으니 '무서워서 못 고친다'는 말이 사라지는 거예요.

→ 넘어가며: "쓰는 법은 허무할 정도로 간단합니다 — 처음 5클릭, 이후 0클릭."

처음 5클릭, 이후 0클릭

⌚ 3분

★ KEY POINT

GitHub 로그인 → Add New → Import → Deploy → URL. 이후엔 push가 전부.

■ 화면

5클릭 스텝. (실행은 실습 Part 9)

💬 이렇게 말하세요

무대 연결은 처음 한 번뿐입니다. vercel.com에서 Continue with GitHub — 로그인하는 순간 연동도 끝나요. Add New, Project, 내 저장소 옆 Import, Deploy. 1분쯤 지나면 폭죽 화면과 함께 URL이 나옵니다. 끝이에요. 이후로는 Vercel 사이트에 들어갈 일 자체가 별로 없습니다 — push가 전부니까. 이따 실습에서 직접 하고, 마지막에 이 0클릭의 위력을 다 같이 관람할 겁니다.

📌 참고

무료(Hobby)는 개인·비사업 학습에 충분. 요금 정책은 수업 직전 한 번 확인해두기.

→ 넘어가며: "자, 마지막 집이자 오늘의 주인공 — 참고."

공유 링크 = DB의 존재 증명

⌚ 3분

★ KEY POINT

친구 폰에 내 결과가 보이려면 데이터가 제3의 장소에 있어야 한다 — 그곳이 DB.

■ 화면

"제3의 장소" 큰 문장 + localStorage 감옥 콜아웃.

💬 이렇게 말하세요

오프닝의 그 장면으로 돌아갑시다. 제 결과 링크를 ㅇㅇ님이 열었더니 제 결과가 보였죠. 논리적으로 따져보세요 — 그 데이터가 제 폰에만 있다면 ㅇㅇ님 폰에 보일 방법이 없습니다. ㅇㅇ님 폰에 있을 리도 없고요. 폰 적이 없으니까. 그럼 답은 하나예요. 데이터가 제 폰도, ㅇㅇ님 폰도 아닌 제3의 장소에 있는 겁니다. localStorage로는 기술이 부족한 게 아니라 논리적으로 불가능한 거예요. 그 제3의 장소가 데이터베이스, 오늘 우리가 지을 참고입니다.

💡 강사 팁

이 슬라이드는 천천히. "논리적으로 불가능"이라는 표현을 살리면 DB의 필요성이 암기가 아니라 추론으로 박힌다.

→ 넘어가며: "근데 왜 하필 DB냐 — 그냥 파일에 쓰면 안 되나?"

왜 파일이 아니라 DB인가 — 테이블 = 규칙 있는 엑셀

⌚ 3분

★ KEY POINT

동시성·검색·규칙 — 파일은 셋 다 무너진다. 테이블은 만 명이 동시에 써도 안 깨지는 엑셀.

■ 화면

3카드(동시성·검색·규칙) + 테이블=엑셀 콜아웃.

💬 이렇게 말하세요

"그냥 파일에 저장하면 안 되나요?" 좋은 질문인데, 세 가지가 무너집니다. 첫째 동시성 — 100명이 동시에 결과를 저장하면 파일은 서로 덮어써요. DB는 줄을 세워서 한 건도 안 잃습니다. 둘째 검색 — "id가 abc123인 결과 하나만 줘"를 파일에선 전체를 읽어야 찾지만 DB는 즉시 꺼냅니다. 셋째 규칙 — "이 칸은 비면 안 됨" 같은 규칙을 DB가 대신 지켜줘요. 생김새는 엑셀이랑 똑같습니다. 열이 항목이고, 한 줄이 응답 하나. 규칙 있는 엑셀, 안 깨지는 엑셀이라고 생각하시면 됩니다.

→ 넘어가며: "이 참고에 말을 걸려면 언어가 필요한데 — 동사가 딱 4개입니다."

SQL 동사 4개 + 우리 테이블(results) 설계

⌚ 3분

★ KEY POINT

INSERT·SELECT·UPDATE·DELETE — 오늘 앱은 앞의 2개만 쓴다. id가 곧 공유 링크가 된다.

■ 화면

동사 4개 태그 + results 테이블 미니카드.

💬 이렇게 말하세요

SQL은 참고에 말 거는 표준 언어인데, 동사가 네 개뿐입니다. 넣기, 꺼내기, 고치기, 지우기. 심리테스트는 결과를 넣고 꺼내기만 하면 되니까 오늘은 앞의 두 개만 씁니다. 우리 테이블 설계를 보세요 — id는 저장할 때마다 자동으로 발급되는 고유번호인데, 이게 그대로 공유 링크의 ?id=가 됩니다. result는 유형 이름 — 통계용이고요. payload엔 결과 전체가 자유 형식으로 들어갑니다. 참, Supabase 속은 PostgreSQL이라고, 대기업들이 쓰는 엔진 그대로예요. 장난감 아닙니다. 그리고 오늘 SQL은 제가 드리는 스크립트 복붙 한 방이면 끝나요.

📌 참고

2차 수강생에게: "테이블 스키마가 곧 데이터의 I/O 계약 — 포·필·범·메 기억나시죠?"

→ 넘어가며: "참고가 인터넷에 있다는 건 — 아무나 문을 두드릴 수 있다는 뜻이죠. 보안 갑니다."

열쇠 2개와 문지기 RLS (보안)

🕒 4분

★ KEY POINT

`anon` = 손님용 방문증(공개 OK, 문지기가 지킴) / `service_role` = 마스터키(절대 공개 금지).

■ 화면

열쇠 2개 2단 + 문지기 규칙표 콜아웃.

💬 이렇게 말하세요

Supabase는 열쇠를 두 개 줍니다. 이거 헛갈리면 사고 나니까 집중해주세요. `anon` 키는 손님용 방문증이에요. 우리 코드에 넣고, GitHub에 공개돼도 됩니다. 왜냐 — 이 방문증으로 뭘 할 수 있는지는 문지기가 정하거든요. 그 문지기가 RLS, 출입 규칙표입니다. 오늘 규칙은 "누구나 넣기와 읽기는 된다, 고치기와 지우기는 아무도 안 된다" — 그래서 방문증이 공개돼도 남의 결과를 조작할 수 없어요. 반대로 `service_role` 키는 마스터키입니다. 문지기를 무시하고 다 할 수 있어요. 이건 절대, 절대 코드나 GitHub에 올리면 안 됩니다. 오늘은 꺼내지도 않을 거예요. 습관 하나만 가져가세요 — 공개돼도 되는 열쇠인지 모르겠으면, 일단 공개하지 마라.

→ 넘어가며: "이제 참고 개설 절차 — 4단계면 끝납니다."

참고 개설 4단계 + 자동 API

⌚ 2분

★ KEY POINT

프로젝트 생성 → SQL 복붙 → 열쇠 복사 → 연결. 테이블을 만들면 넣기/꺼내기 창구(API)가 자동 생성.

■ 화면

4단계 스텝 + 무료 일시정지 경고.

💬 이렇게 말하세요

절차는 네 단계입니다. 프로젝트 만들고 — 리전은 서울로. SQL 에디터에 제 스크립트 붙여넣고 Run — 클릭으로 만들면 10분 걸릴 걸 10초에 끝냅니다. 설정에서 열쇠 두 값 복사하고, Claude Code에게 넣어달라고 하면 끝. 그리고 Supabase의 좋은 점 — 테이블을 만들면 '넣기/꺼내기 창구'가 자동으로 생깁니다. 은행 창구가 저절로 생기는 거예요. 코드에선 한 줄로 부릅니다. 아, 중요한 공지 — 무료 참고는 일주일쯤 안 쓰면 잠들어요. 몇 주 뒤에 "제 테스트가 안 열려요" 하시면 십중팔구 이겁니다. 대시보드에서 Restore 한 번이면 깨어나요.

→ 넘어가며: "이론 마지막 — 세 집을 한 장에 모읍시다."

마스터 그림 — 세 집이 한 몸이 되는 순간

⌚ 4분

★ KEY POINT

이 그림 어디에도 '켜져 있는 내 컴퓨터'는 없다 — 그래서 내 컴퓨터를 꺼도 서비스는 산다.

■ 화면

내 컴퓨터 → push → GitHub → 자동배포 → Vercel ↔ Supabase 연결도.

💬 이렇게 말하세요

오늘의 마스터 그림입니다. 이 한 장만 남으면 오늘 수업 성공이에요. 내 컴퓨터의 역할은 '만들고 고치는 것'뿐입니다. 고치면 push — 기억(GitHub)에 쌓이고, 벨트를 타고 무대(Vercel)에 오르고, 접속자들이 무대의 앱을 통해 창고(Supabase)에 데이터를 넣고 꺼냅니다. 자, 이 그림을 손가락으로 짚어보세요 — 어디에도 '켜져 있는 내 컴퓨터'가 없습니다. 그래서 제가 지금 이 노트북을 덮어도 서비스는 삽니다. 그게 오늘 제목의 뜻이에요.

💡 강사 팁

"손가락으로 짚어보세요"는 실제로 시키기. 스스로 '내 컴퓨터가 없다'를 발견하게 하는 게 포인트.

→ 넘어가며: "그림 검증 — 친구가 링크를 여는 순간을 추적해봅시다."

데이터의 여행 + 역할 확인 문답

⌚ 4분

★ KEY POINT

한 번의 클릭에 세 집이 다 움직인다. 문답 3개로 역할 분담을 수강생 입으로 말하게 한다.

■ 화면

여행 3단계 타임라인 + 문답 3카드.

💬 이렇게 말하세요

친구가 제 공유 링크를 엽니다. 무슨 일이 벌어지나 — 먼저 친구 폰이 무대에 요청해서 앱 화면을 받아요. 그 화면 속 코드가 링크의 id를 들고 창고에 가서 select — 제 결과를 꺼내옵니다. 그게 친구 폰에 그려지는 거예요. 자 그럼 퀴즈. 코드를 잘못 고쳐 망가뜨렸다, 어느 집에 가죠? (기억 — 커밋 복귀, 또는 무대 — 롤백요.) 테스트가 대박 나서 1만 명이 몰렸다, 누가 일하죠? (무대랑 창고요. 내 컴퓨터 아님!) 응답 데이터를 엑셀로 뽑고 싶다? (창고 — Table Editor.) 완벽합니다. 이제 원리는 끝 — 지금부터 이 그림을 여러분 손으로 만듭니다.

📌 참고

✂ 2회 분할 시 여기서 1회차 종료. 숙제: 계정 3종 확인 + 테스트 주제 후보 2~3개 생각해오기.

→ 넘어가며: "실습 시작 — 먼저 여러분의 테스트를 기획합니다."

내 테스트 기획 — Claude와 대화 → 기획안.md

⌚ 15분

★ KEY POINT

자유 기획. 최소 요건 2개만 — 유형 개념(통계) + 공유 기능. 산출물은 기획안.md 저장까지.

■ 화면

4단계 스텝. 수강생: claude.ai + 배포된 '기획 프롬프트'(03_테스트기획_프롬프트.txt).

💬 이렇게 말하세요

배포해드린 기획 프롬프트를 claude.ai 새 대화에 통째로 붙여넣으세요. 그러면 Claude가 취향을 두 개 묻고, 주제 후보 다섯 개를 제안합니다. 나는 무슨 동물인지, 어떤 음식인지, 방에 들어갔더니 뭐가 있는지 — 뭐든 좋아요. 마음에 드는 걸 고르거나 섞으세요. 그다음 성향 축, 결과 유형, 질문 순서로 같이 설계해나가다가, 마음에 들면 "완성"이라고 치세요. 기획안이 한 장으로 나옵니다. 그걸 키트 폴더의 기획안.md에 저장하는 것까지가 이번 파트예요 — Claude Code에게 "이 내용으로 기획안.md 만들어줘" 하고 붙여넣는 게 제일 편합니다. 참, 이 프롬프트 구조 어디서 봤죠? 역할·목적·맥락·제약·출력 — 1차 R-PCCO입니다.

💡 강사 팁

"질문 10개짜리 대작"을 기획하는 수강생이 꼭 있다 — 막지 말 것(자유가 이 방식의 핵심). 대신 "구현은 다음 파트에서 30분"이라고 시간 감각만 잡아주기.

→ 넘어가며: "설계도가 나왔으니 — 짓습니다."

백엔드 키트 위에 바이브 코딩

⌚ 30분

★ KEY POINT

구현 요청은 한 줄 — "기획안.md 읽고 CLAUDE.md 규칙 지켜서 구현해줘." 공유 버튼 회색이 '정상'.

■ 화면

구현 요청 미니카드 + 울타리·계약·회색버튼 3카드. 수강생: 키트 폴더에서 claude 실행.

💬 이렇게 말하세요

키트 폴더에서 claude를 실행하고, 이 한 줄을 붙여넣으세요 — "기획안.md를 읽고, CLAUDE.md의 규칙을 지켜서 구현해줘." 끝입니다. 어떻게 한 줄로 되냐 — 폴더에 CLAUDE.md라는 규칙서가 있거든요. 6차에서 배운 'AI한테 한 장'이 실전에서 일하는 순간입니다. 여러분의 기획이 뭐든, 참고 출력은 db.js의 문 세 개로만 하도록 규칙이 잡아줍니다. 완성되면 브라우저로 열어서 내 기획대로 도는지 확인하세요. 공유 버튼이 회색으로 '참고 연결 대기 중'이라고 떠 있을 텐데 — 그게 정상입니다. 아직 참고가 없잖아요. 이따 켜지는 순간이 옵니다.

💡 강사 팁

순회하며 막힌 수강생 체크. 15분 넘게 꼬이면 참고샘플(동물테스트)로 갈아타게 하고 "내 기획 구현은 숙제"로 — 뒷단 진도가 우선.

→ 넘어가며: "내 폴더에만 있는 이 앱 — 첫 번째 집에 입주시키시다."

GitHub — 기억의 집 입주

🕒 12분

★ KEY POINT

로그인 → 올리기 → 웹에서 확인 → 하나 고치고 push. 기획안.md도 함께 박제된다.

■ 화면

타임라인 4단계. 수강생: Claude Code에 3동작 + 브라우저로 repo 확인.

💬 이렇게 말하세요

"GitHub에 로그인해줘" — 브라우저 인증만 직접 해주시고요. 되면 "이 폴더를 GitHub 새 저장소로 올려줘, 이름은 my-test." 잠시 뒤 주소가 나오면 브라우저로 열어보세요. 여러분 파일들이 보이죠? 방금 여러분 코드가 지구 반대편 데이터센터에 박제됐습니다. 기획안.md도 같이 올라간 것 보이세요? 기획의 역사까지 기억에 남는 거예요. 이제 연습 한 번 — 앱 제목이든 색이든 하나만 고치고 "커밋하고 push해줘." commits에 두 번째 줄이 생기고, 클릭하면 지운 줄은 빨강게 넣은 줄은 초록으로 — 이게 여러분 코드의 역사책입니다.

→ 넘어가며: "기억 완료. 이제 무대에 세웁니다."

Vercel — 무대에 서다 (5클릭)

🕒 10분

★ KEY POINT

코드 작업 없음 — 전부 대시보드. URL이 나오면 폰에서 열어본다.

■ 화면

5클릭 스텝. 수강생: vercel.com 대시보드.

💬 이렇게 말하세요

vercel.com — Continue with GitHub로 로그인하세요. 로그인이 곧 연동입니다. Add New, Project — 방금 만든 my-test가 목록에 보일 거예요. 안 보이면 GitHub 권한 창에서 저장소를 허용해주면 됩니다. Import, 그리고 설정은 그대로 두고 Deploy. 1분만 기다리면 — 폭죽과 함께 주소가 나옵니다. 그 주소를 폰에서 열어보세요. 여러분이 기획하고 만든 테스트가, 여러분 컴퓨터가 아니라 무대 위에서 돌아가고 있습니다. "7차랑 뭐가 달라요?" — 아직 안 보이죠. 마지막 파트에서 보여드립니다.

→ 넘어가며: "이제 오늘의 산 — 창고입니다."

Supabase — 창고를 잇다 (4단계)

🕒 25분

★ KEY POINT

Table Editor에 내 응답이 꽂히는 순간이 와우 ①. 그리고 공유 버튼이 켜진다.

■ 화면

4단계 스텝. 수강생: supabase.com + 키트의 supabase_setup.sql + Claude Code.

💬 이렇게 말하세요

supabase.com에서 New Project — 이름은 자유, 리전은 꼭 서울(Northeast Asia)로. DB 비밀번호는 만들어서 메모만 해두세요, 오늘 다시 쓸 일은 없습니다. 생성되는 1~2분 동안 복습 — 테이블이 뭐라고요? 규칙 있는 엑셀. 됐으면 왼쪽 SQL Editor에 키트의 supabase_setup.sql 내용을 통째로 붙여넣고 Run. Success 한 줄이면 테이블과 문지기 규칙까지 완성입니다. Table Editor에서 빈 results 테이블 확인하시고요. 이제 Settings의 API에서 Project URL과 anon public key를 복사해서, Claude Code에게 "config.js에 이 열쇠 넣어줘" 하고 붙여넣으세요. 그리고 로컬에서 테스트를 한 번 끝까지 풀어보세요. 자 — Supabase 화면으로 돌아가서 Table Editor를 새로고침하면... 여러분 응답이 한 줄 꽂혀 있습니다. 방금 여러분 데이터가 창고에 들어간 거예요. 그리고 보세요 — 공유 버튼이 켜졌습니다.

💡 강사 팁

에러의 90%는 열쇠 복사 실수와 SQL 미실행. "브라우저 콘솔 에러를 그대로 Claude Code에 붙여넣어"가 만능 처방.

→ 넘어가며: "근데 이 변경사항, 아직 무대엔 없죠? — 맥박 갑니다."

맥박 확인 + 공유 이벤트 (와우 3연타)

⌚ 13분

★ KEY POINT

push → 자동 재배포 관람(와우 ②) → 링크 교환 → 남의 폰에서 내 결과(와우 ③).

■ 화면

타임라인 4단계. 수강생: Claude Code + Vercel Deployments + 폰.

💬 이렇게 말하세요

Claude Code에 "커밋하고 push해줘." 그리고 다 같이 Vercel의 Deployments 화면을 보세요. 새 줄이 생겼죠 — Building... 그리고 Ready. 저는 Vercel에 손도 안 댔습니다. push가 방아쇠예요. 이게 7차와의 차이, 맥박입니다. 자, 이제 폰에서 여러분 테스트를 풀고, 공유 링크를 복사해서 옆 사람과 교환하세요. ...어때요? 남의 폰에서 여러분 결과가 열리죠. 아까 이론에서 추론했던 그 '제3의 장소'가 지금 눈앞에서 증명된 겁니다. 통계도 보세요 — 참여자가 늘 때마다 퍼센트가 바뀝니다. 창고가 살아있다는 뜻이에요. 마지막으로 단톡방에 각자 URL 올리고, 서로 테스트 풀어줍니다!

→ 넘어가며: "정리하겠습니다 — 오늘의 다섯 단어."

5요소 회수 + 습관 3가지 + 공지

⌚ 7분

★ KEY POINT

"코드는 GitHub에, 실행은 Vercel에, 데이터는 Supabase에 산다. 셋 다 내 컴퓨터가 아니다."

■ 화면

습관 3 리스트 + 공지 콜아웃.

💬 이렇게 말하세요

오늘의 다섯 단어 — 몸, 기억, 무대, 창고, 맥박. 문장 하나로 줄이면: 코드는 GitHub에 살고, 실행은 Vercel에 살고, 데이터는 Supabase에 산다. 셋 다 내 컴퓨터가 아니다 — 그래서 내 컴퓨터를 꺼도 서비스는 산다. 습관 세 개 가져가세요. 고치면 커밋. 마스터키는 절대 올리지 않기. 배포했으면 폰에서 확인. 공지 둘 — 무료 창고는 일주일 안 쓰면 잠들어요, Restore로 깨우면 됩니다. 그리고 심화 숙제: 7차에서 배운 PWA 5요소를 이 테스트에 엮어보세요. 단, 시작 전에 "이 서비스는 왜 웹이 정답이었나"부터 답해보고요 — 할 일 앱은 매일 여는 도구라 설치가 정답이었지만, 심리테스트는 링크로 한 번 방문하는 서비스입니다. 무엇을 만드느냐가 형태를 정합니다.

→ 넘어가며: "마지막 한 장."

"공유 링크가 되는 순간, 앱이 아니라 서비스다"

03분

★ KEY POINT

오늘 배운 건 심리테스트가 아니라 — 무엇을 만들든 세상에 내보내는 표준 배관.

■ 화면

클로징(다크) — 핵심 문장 + 다음 예고.

💬 이렇게 말하세요

여러분은 오늘 심리테스트를 만든 게 아닙니다. 아이디어가 뭐든 — 리더보드든 방명록이든 예약 시스템이든 — 그걸 세상에 내보내는 표준 배관을 손에 넣은 거예요. 기억에 올리고, 무대에 잇고, 창고를 연결한다. 이 세 동작은 이제 여러분 겁니다. 다음 시간엔 이 배관 위에 뭘 얹을지 — 기대하셔도 좋습니다. 오늘 만든 테스트, 꼭 친구들한테 뿌려보세요. 창고에 쌓이는 진짜 응답이 오늘 최고의 복습입니다. 수고하셨습니다!